



We deliver cybersecurity expertise

XMLABS

Cassage de mots de passe
Focus : John & Hashcat

avril 26

Concepts

Cassage de mots de passe

1

Concepts

<https://passwordvillage.org/general.html>

Hashing

- Fonction à sens unique
- Le seul moyen de retrouver la valeur originelle c'est de tester une valeur d'entrée et d'utiliser le même algorithme
- Longueur de sortie fixe
- Ex: MD5, SHA1, BCrypt, PBKDF2-HMAC-SHA256, BCODE, ...

Encryption

- Réversible avec une clé
- Longueur variable selon l'entrée

TL;DR

Encryption is reversable, hashing is not



```
1 echo -n "kevin" | md5sum
2 9d5e3ecdeb4cdb7acfd63075ae046672 -
```

Concepts

<https://passwordvillage.org/general.html>

Salting

- Ajoute une chaîne de caractère à notre mot de passe avant de le hacher
- Peut être placé avant ou après dans le hash avec un séparateur ou non selon le format
- Fait en sorte que deux mots de passe identiques produisent une sortie différente
- Si 10 utilisateurs ont le même mot de passe, il faut faire 10 tests au lieu d'un seul sans sel
- Consomme plus en termes de calcul



```
b353977827f67a4ae0318f3a9447fae1c13d9d90:b8d18ca
|-----| | |-----|
|               | | salt
|               | |
|               | | separator
```

b8d18capassword

Concepts

<https://passwordvillage.org/general.html>

Itérations

- Principe : on hash un mot de passe, puis on re-hash le résultat, puis on le re-hash, etc., etc.
- Certains algorithmes utilisent un nombre fixe d'itérations :
 - md5crypt : utilise l'algorithme MD5 et 1 000 itérations
 - sha512crypt : utilise l'algorithme SHA512 avec 5 000 itérations

Collision

- C'est quand deux entrées distinctes produisent en sortie le même hash = collision
- Pas besoin de casser le mot de passe puisqu'une autre entrée permettra de provoquer une vérification du hash réussie.

Identifier un Hash

Identifier un hash

- Parfois la réponse est dans le hash
 - `$racf$*USER*FC2577C6EBE6265B`
 - `SCRYPT:1024:1:1:MDlwMzMwNTQwNDQyNQ==:5FW+zWivLxgCWj7qLiQbeC8zaNQ+qdO0NUinvqyFcfo=`
 - `{x-issaha, 1024}C0624EvGSdAMCtuWnBBYBGA0chvqAflKY74oEpw/rpY=`
- Regardez la longueur du hash (hexa)
 - `echo -n "8743b52063cd84097a65d1633f5c74f5" | wc -c` -> **32** chars = **MD5**
 - `echo -n "fffb4761cba839470133bee36aeb139f58d7dbaa9" | wc -c` -> **40** chars = **SHA1**

Ressources utiles

- https://hashcat.net/wiki/doku.php?id=example_hashes
- <https://openwall.info/wiki/john/sample-hashes>
- <https://github.com/blackploit/hash-identifier>

Fichier de hash

Format d'un fichier de hash

[USERNAME]:HASH

Exemple de format de fichier valide :

JCOUPPEY:4cc29cd1628c66f107397116bd24725f

ou

4cc29cd1628c66f107397116bd24725f

Deux modes d'attaques

Attaque par dictionnaire

On utilise un dictionnaire composé de candidats potentiels, et on regarde si notre hash correspond au mot de passe indiqué

Attaque par brute-force

On teste toutes les possibilités de mots de passe en utilisant un jeu de caractère spécifique (chiffres, lettres minuscules, lettres majuscules, caractères hexadécimaux, caractères spéciaux, tout)

Règle

Mangling Rule = règle de transformation

- Décrit la façon dont l'outil de cassage doit transformer chaque candidat de mot de passe fourni.
- Exemple : la règle "u" (**u**ppercase) va prendre chaque mot de la wordlist et le mettre en majuscule.
 - Si le mot est "xmco", il deviendra "XMCO"
 - Si le mot est "XMco", il deviendra "XMCO"
- Il est possible de chaîner plusieurs règles entre-elles, par exemple "**u**" et "**r**" (**r**everse) vont mettre en majuscule puis inverser les lettres :
 - Si le mot est "xmco", il deviendra "OCMX"
- La plupart des règles sont supportées à la fois par hashcat et john mais il peut y avoir des règles spécifiques à chacun d'eux.

John The Ripper

2

John

<https://www.openwall.com/john/doc/>

⚠ La version jumbo est à préférer (version communautaire enrichie et plus poussée en termes de hash supportés, règles, etc.)

Ligne de commande simple

```
> john hashfile.txt
```

Description

- Reconnaît automatiquement le format du hash basé sur plusieurs critères (longueur, préfixe, charset, etc.)
- Plusieurs formats peuvent être identifiés si ambiguïté (ex: Raw-MD5 == LM), un seul sera choisi selon des heuristiques particulières, sinon il faut le forcer avec `--format=[FORMAT]`
- `--list=formats` : liste tous les formats de hash pris en charge (aussi `--list=subformats`)
- Pensez à supprimer régulièrement son fichier `$HOME_JOHN/john.log`

John

<https://www.openwall.com/john/doc/>

Ligne de commande classique

```
> john hashfile.txt --format=[FORMAT] --wordlist=[WORDLIST] --rules=[RULES]  
> john hashfile.txt --format=[FORMAT] -w=[WORDLIST] -ru=[RULES]
```

Les options essentielles

- `--format=[NAME]` : forcer le format du HASH (ex: raw-md5, raw-sha1, oracle, etc.)
- `--wordlist=[FICHER]` : utiliser une wordlist (***rockyou.txt***, ou autre)
- `--rules=[RULE]` : utiliser une règle (spécifiée dans ***john.conf***)

John

Single mode (--single)

- Le mode recommandé avant de faire quoi que ce soit
- Applique tout un tas de règles sur le username pour trouver le mot de passe

Incremental mode (--incremental)

- Bruteforce incrémental (aaaa aaab aaac ...)

External mode (--external=[RULE])

- Coder son propre programme dans un pseudo-C pour dire à john comment il doit générer ses candidats
- -ex=leet : règle LeetCode (une règle hashcat existe aussi)

John

Autres options

- **--stdin** : lit une wordlist depuis l'entrée standard
- **--stdout** : affiche toutes les combinaisons testées sur la sortie standard
 - 💡 très utile si on a besoin de faire des listes pour bruteforcer un service
 - 💡 ou juste pour voir ce que fait une règle et génère comme output

Useless info

- Y a une version GUI : <https://openwall.info/wiki/john/johnny>

Sans options de cassage particulier

```
[jcouppey@XMC0-JCOU /tmp] on ☁ (eu-west-1) took 2s
> john _ --format=raw-md5 --pot=p
Using default input encoding: UTF-8
Loaded 3 password hashes with no different salts (Raw-MD5 [MD5 128/128 ASIMD 4x2])
Cracked 2 password hashes (are in p), use "--show"
Remaining 1 password hash
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/opt/Crypto/john/run/password.lst
Enabling duplicate candidate password suppressor using 256 MiB
Disabling duplicate candidate password suppressor
0g 0:00:00:15 3.20% 2/3 (ETA: 15:22:52) 0g/s 10124Kp/s 10124Kc/s 10124KC/s pajore..p1o2a3o4
0g 0:00:00:25 7.04% 2/3 (ETA: 15:20:59) 0g/s 14151Kp/s 14151Kc/s 14151KC/s 9valentino1..9230786
0g 0:00:00:36 11.16% 2/3 (ETA: 15:20:26) 0g/s 15502Kp/s 15502Kc/s 15502KC/s tazisa..wezewolf15
0g 0:00:03:59 85.66% 2/3 (ETA: 15:19:43) 0g/s 17910Kp/s 17910Kc/s 17910KC/s 045888599..natalunia599
0g 0:00:04:36 DONE 2/3 (2026-03-20 15:19) 0g/s 17926Kp/s 17926Kc/s 17926KC/s Charmar138..Sambarock38
Proceeding with incremental:ASCII
0g 0:00:04:43 3/3 0g/s 17838Kp/s 17838Kc/s 17838KC/s 5uv..samsa180
9a1b3d (?)
1g 0:00:05:01 DONE 3/3 (2026-03-20 15:20) 0.003318g/s 18764Kp/s 18764Kc/s 18764KC/s 9a1msm..9a1kbb
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed
```

John

<https://www.openwall.com/john/doc/>

Scripts / Plugins

```
> ls $JOHN_HOME/run/*2john*
```

```
1password2john.py
```

```
7z2john.pl
```

```
...
```

```
keepass2john
```

```
telegram2john.py
```

```
...
```

```
rar2john
```

```
zip2john
```

```
...
```

Description

- Scripts qui permettent **d'extraire automatiquement des hash** pour pouvoir ensuite les casser avec John
 - Une archive chiffrée : on extrait le hash en utilisant 7z2john.pl, zip2john



Attention parfois les formats peuvent ne pas être bien pris en charge
(RETEX: zip chiffré ou bitlocker)

John

Wordlist mode

- Un mot de passe à tester par ligne
- Si on active une **mangling** rule ("règle de transformation"), toutes les règles seront appliquées à chaque ligne
- ⚠️ Aucun traitement préalable (tri ou suppression des doublons) n'est appliqué par john
- Selon la doc, c'est mieux si la wordlist est triée par ordre alphabétique car john est plus performant si chaque candidat lu à la suite diffère de quelques caractères

The recommended way to sort a wordlist for use with default wordlist rule set is:

```
tr A-Z a-z < OLD_WORDLIST | sort -u > NEW_WORDLIST
```

Rules

<https://www.openwall.com/john/doc/RULES.shtml>

Simple commands

- `l` convert to lowercase
- `u` convert to uppercase
- `c` capitalize
- `r` reverse: "Fred" -> "derF"
- `d` duplicate: "Fred" -> "FredFred"
- `$X` append character X to the word
- `^X` prefix the word with character X
- `AN"STR"` insert string STR into the word at position N

Rules

<https://www.openwall.com/john/doc/RULES.shtml>

Simple commands

- **l** convert to lowercase
- **u** convert to uppercase
- **c** capitalize
- **r** reverse: "Fred" -> "derF"
- **d** duplicate: "Fred" -> "FredFred"
- **\$X** append character X to the word
- **^X** prefix the word with character X
- **AN"STR"** insert string STR into the word at position N

Fichier : john.conf

[List.Rules:XMC0]

^A\$0\$1\$2\$3

Fichier : w

ceci

est

une

wordlist

john -w=w -ru=xmco --stdout
?

Rules

<https://www.openwall.com/john/doc/RULES.shtml>

Simple commands

- **l** convert to lowercase
- **u** convert to uppercase
- **c** capitalize
- **r** reverse: "Fred" -> "derF"
- **d** duplicate: "Fred" -> "FredFred"
- **\$X** append character X to the word
- **^X** prefix the word with character X
- **AN"STR"** insert string STR into the word at position N

Fichier : john.conf

[List.Rules:XMC0]

^A\$0\$1\$2\$3

Fichier : w

ceci

est

une

wordlist

john -w=w -ru=xmco --stdout

Aceci0123

Aest0123

Aune0123

Awordlist0123

Rules

<https://www.openwall.com/john/doc/RULES.shtml>

Simple commands

- **l** convert to lowercase
- **u** convert to uppercase
- **c** capitalize
- **r** reverse: "Fred" -> "derF"
- **d** duplicate: "Fred" -> "FredFred"
- **\$X** append character X to the word
- **^X** prefix the word with character X
- **AN"STR"** insert string STR into the word at position N

Fichier : john.conf

[List.Rules:XMCO]

AB"test"

Fichier : w

rezorijezor

zerlkjzlerkjzr

zelrkjrlkzjer

zlerjzeorzjre

john -w=w -ru=xmco --stdout

?

Rules

<https://www.openwall.com/john/doc/RULES.shtml>

Simple commands

- **l** convert to lowercase
- **u** convert to uppercase
- **c** capitalize
- **r** reverse: "Fred" -> "derF"
- **d** duplicate: "Fred" -> "FredFred"
- **\$X** append character X to the word
- **^X** prefix the word with character X
- **AN"STR"** insert string STR into the word at position N
- **⚠** Si les positions sont > 9, on utilise des caractères hexadécimaux > 9 (A = 10, B = 11, etc.)

Fichier : john.conf

```
[List.Rules:XMCO]  
AB"test"
```

Fichier : w

```
rezorijezor  
zerlkjzlerkjzr  
zelrkjrlkzjer  
zlerjzeorzjre
```

```
john -w=w -ru=xmco --stdout  
rezorijezortest  
zerlkjzlerktestjzr  
zelrkjrlkzjtester  
zlerjzeorzjtestre
```

John

Afficher les mots de passe cassés

```
> john hashfile.txt --format=[FORMAT] --show
```

⚠ Il ne faut pas d'autres options de cassage mentionnées (-w ou -ru)

Hashcat

3

Hashcat

<https://hashcat.net/hashcat/>

Exemple de ligne de commande

```
> hashcat -m [HASH_ID] -a [ATTACK] hashfile.txt [OPTIONS_DEPENDANTES_DU_MODE_D'ATTAQUE_CHOISI]
```

Hashcat

<https://hashcat.net/hashcat/>

Définir le type de hash (-m [ID])

La bible : https://hashcat.net/wiki/doku.php?id=example_hashes

Generic hash types

Hash-Mode	Hash-Name	Example
0	MD5	8743b52063cd84097a65d1633f5c74f5
10	md5(\$pass.\$salt)	01dfae6e5d4d90d9892622325959afbe:7050461
20	md5(\$salt.\$pass)	f0fda58630310a6dd91a7d8f0a4ceda2:4225637426
30	md5(utf16le(\$pass).\$salt)	b31d032cfdcf47a399990a71e43c5d2a:144816
40	md5(\$salt.utf16le(\$pass))	d63d0e21fdc05f618d55ef306c54af82:13288442151473
50	HMAC-MD5 (key = \$pass)	fc741db0a2968c39d9c2a5cc75b05370:1234
60	HMAC-MD5 (key = \$salt)	bfd280436f45fa38eaacac3b00518f29:1234
70	md5(utf16le(\$pass))	2303b15bfa48c74a74758135a0df1201
100	SHA1	b89eaac7e61417341b710b727768294d0e6a277b
110	sha1(\$pass.\$salt)	2fc5a684737ce1bf7b3b239df432416e0dd07357:2014
120	sha1(\$salt.\$pass)	cac35ec206d868b7d7cb0b55f31d9425b075082b:5363620024
130	sha1(utf16le(\$pass).\$salt)	c57f6ac1b71f45a07dbd91a59fa47c23abcd87c2:631225
140	sha1(\$salt.utf16le(\$pass))	5db61e4cd8776c7969cfd62456da639a4c87683a:8763434884872

Hashcat

Attack Mode (-a)

- 0 | **Straight** : attaque par dictionnaire via une *wordlist* à laquelle on peut appliquer ou non des *rules*
- 1 | **Combination** : combine 2 wordlists où via list1.txt (mots) et list2.txt (suffixes)
- 3 | **Bruteforce** : génère des caractères selon un charset défini via un masque
- 6 | **Hybrid Wordlist + Mask** : on utilise une wordlist ET on ajoute un masque à la fin de chaque ligne
- 7 | **Hybrid Mask + Wordlist** : utilise un masque ET ajoute la chaque ligne de la wordlist
- 9 | **Association** : associe un contexte pour casser un hash en utilisant un hint correspondant

Hashcat

https://hashcat.net/wiki/doku.php?id=dictionary_attack

Attack Mode (-a 0)

0 | Dictionary :

Attaque par dictionnaire via une *wordlist* à laquelle on peut appliquer ou non des *rules*

```
hashcat -m 0 -a 0 hash.txt wordlist.txt
```

```
hashcat -m 0 -a 0 hash.txt wordlist.txt -r best64.rule
```

Hashcat

https://hashcat.net/wiki/doku.php?id=combinator_attack

Attack Mode (-a 1)

1 | Combination : combine 2 wordlists

```
hashcat -m 0 -a 1 hash.txt list1.txt list2.txt
```

list1.txt -> les mots de passe à tester

list2.txt -> les suffixes à ajouter à chaque ligne

-j: règle appliquée à chaque mot de la première wordlist

-k: règle appliquée à chaque mot de la deuxième wordlist

dict1: xmco

dict2: 2021
2020

```
hashcat -m 0 -a 1 hash.txt dict1.txt dict2.txt -j '$-' -k '$!'  
?
```

Hashcat

https://hashcat.net/wiki/doku.php?id=combinator_attack

Attack Mode (-a 1)

1 | Combination : combine 2 wordlists

```
hashcat -m 0 -a 1 hash.txt list1.txt list2.txt
```

list1.txt -> les mots de passe à tester

list2.txt -> les suffixes à ajouter à chaque ligne

-j: règle appliquée à chaque mot de la première wordlist

-k: règle appliquée à chaque mot de la deuxième wordlist

dict1: xmco

dict2: 2021
2020

```
hashcat -m 0 -a 1 hash.txt dict1.txt dict2.txt -j '$-' -k '$!'
```

xmco-2020!

xmco-2021!

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

```
hashcat -m 0 -a 3 hash.txt '?d?d?d?d?d?d?d?d'
```

```
hashcat -m 0 -a 3 hash.txt '?d?d?d?d?d?d?d?d' --increment
```

```
hashcat -m 0 -a 3 hash.txt '?d?d?d?d?d?d?d?d' --increment-min=4
```

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

?l = abcdefghijklmnopqrstuvwxyz

?u = ABCDEFGHIJKLMNOPQRSTUVWXYZ

?d = 0123456789

?h = 0123456789abcdef

?H = 0123456789ABCDEF

?s = «space»!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~

?a = ?l?u?d?s

?b = 0x00 - 0xff

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

?l = abcdefghijklmnopqrstuvwxyz

?u = ABCDEFGHIJKLMNOPQRSTUVWXYZ

?d = 0123456789

?h = 0123456789abcdef

?H = 0123456789ABCDEF

?s = «space»!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~

?a = ?l?u?d?s

?b = 0x00 - 0xff

```
hashcat -m 0 -a 3 hash.txt '?d?d?d?d?d?d?d?d'
```

-> bruteforce un mot de passe de 8 caractères constitués uniquement de chiffres (00000000 à 99999999)

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

?l = abcdefghijklmnopqrstuvwxyz

?u = ABCDEFGHIJKLMNOPQRSTUVWXYZ

?d = 0123456789

?h = 0123456789abcdef

?H = 0123456789ABCDEF

?s = «space»!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~

?a = ?l?u?d?s

?b = 0x00 - 0xff

```
hashcat -m 0 -a 3 hash.txt '?d?d?d?d?d?d?d?d' --increment --increment-min=4
```

-> bruteforce un mot de passe de 4 à 8 caractères constitués uniquement de chiffres (0000 à 99999999)

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

?l = abcdefghijklmnopqrstuvwxyz

?u = ABCDEFGHIJKLMNOPQRSTUVWXYZ

?d = 0123456789

?h = 0123456789abcdef

?H = 0123456789ABCDEF

?s = «space»!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~

?a = ?l?u?d?s

?b = 0x00 - 0xff

```
hashcat -m 0 -a 3 hash.txt '?u?l?l?l?l?d?d?d' --increment --increment-min=4
```

-> brute force un mot de passe dont la première lettre est une majuscule, les 4 suivantes une minuscule et les 3 derniers un chiffre : Aaaa000 -> Zzzz999

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Attack Mode (-a 3)

3 | Mask-Attack : génère toutes les combinaisons par rapport à un masque donné

-1 User-defined charset

...

-8 User-defined charset

```
hashcat -m 0 -a 3 hash.txt -1 '?u?l?d' '?1?1?1?1?1?1?1'
```

-> majuscule / minuscule / chiffre

```
hashcat -m 0 -a 3 hash.txt -1 'Xx' -2 'Mm' -3 'Cc' -4 'Oo' '?1?2?3?4?d?d?d?d'
```

-> teste toutes les casses de la chaine "XMCO"

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Mask File (.hcmask)

Permet de définir plusieurs masques au sein d'un fichier

Filename: **xmco.hcmask**

- Xmco?d?d?d?d
- xmco?d?d?d?d
- XMCO?d?d?d?d

```
hashcat -a 3 -m 0 hash_file xmco.hcmask --increment
```

Hashcat

https://hashcat.net/wiki/doku.php?id=mask_attack

Mask File (.hcmask)

Peut remplacer l'option en ligne de commande -1 [mask] -2 [mask] ...
[?1,] [?2,] [?3,] [?4,] mask

Filename: **xmco2.hcmask**

- ?d?l,test?1?1?1
- abcdef,0123,ABC,789,?3?3?3?1?1?1?1?2?2?4?4?4?4
- ?u?l,?s?d,?1?a?a?a?a?2

```
hashcat -a 3 -m 0 hash_file xmco.hcmask -increment
```

Hashcat

https://hashcat.net/wiki/doku.php?id=hybrid_attack

Attack Mode (-a 6)

6 | Hybrid Wordlist + Mask : combine une wordlist et un masque

```
hashcat -m 0 -a 6 hash.txt wordlist.txt '?d?d?d?s'
```

wordlist.txt:

xmco

Xmco

XMCO

- > bruteforce de **xmco000**[space] à **xmco999~**
- > bruteforce de **Xmco000**[space] à **Xmco999~**
- > bruteforce de **XMCO000**[space] à **XMCO999~**

Hashcat

https://hashcat.net/wiki/doku.php?id=hybrid_attack

Attack Mode (-a 7)

7 | Hybrid Mask + Wordlist : combine un masque et une wordlist

```
hashcat -m 0 -a 7 hash.txt '?d?d?d?s' wordlist.txt --increment
```

wordlist.txt:

xmco

Xmco

XMCO

- > bruteforce de 0[space]xmco à 999~xmco
- > bruteforce de 0[space]Xmco à 999~Xmco
- > bruteforce de 0[space]XMCO à 999~XMCO

Hashcat

https://hashcat.net/wiki/doku.php?id=association_attack

Attack Mode (-a 9)

9 | **Association** : Semblable au mode single de John avec l'utilisation du username comme indice.

Associe un *hint* spécifique à utiliser avec un hash.

Nouveau mode qui existe depuis 2020.

Il faut impérativement un fichier de *hash* et un fichier de *hint* qui contient exactement le même nombre de lignes puisque le hint de la ligne1 sera testé avec le hint de la ligne2, etc (association).

Hashcat

https://hashcat.net/wiki/doku.php?id=association_attack

Comparaison entre le mode *dictionary* et le mode *association*

Création d'une liste de 10 000 hash avec 10 000 hints :

hint.txt

001071
02011511
01papyrus
010108dh
000123258
[...]

hash.txt

\$2a\$05\$KReyKhSyKRi0LBSwLBWyLOj88ZjDCPMD83re/SBNJ9TXJrrPLDbwW
\$2a\$05\$LBCyKhC1MRGvLxeuLxOxMOilYzVzEnWVFr79uLTj6vTPAPO2y2tbS
\$2a\$05\$MB.3MBazLhW0MRKzMBSzKOsHZE77Ou9mGW88yOG2Q5TxF5yvLe48C
\$2a\$05\$LBGyLBeuKxavLh.3MRWyLOhOiQVGYYyUBYE.rOUiRomCQFGIZfutEK
\$2a\$05\$LxiwMR.wKhGvKBiyKxeuM.rHYpUlrIOLksCsvo//8YT5J5z28rzBW
[...]

```
hashcat -m 3200 -a 0 hash.txt hint.txt
```

```
hashcat -m 3200 -a 9 hash.txt hint.txt
```

Hashcat

https://hashcat.net/wiki/doku.php?id=association_attack

Comparaison entre le mode *dictionary* et le mode *association*

Création d'une liste de 10 000 hash avec 10 000 hints :

hint.txt

001071
02011511
01papyrus
010108dh
000123258
[...]

hash.txt

\$2a\$05\$KReyKhSyKRi0LBSwLBWyLOj88ZjDCPMD83re/SBNJ9TXJrrPLDbwW
\$2a\$05\$LBCyKhC1MRGvLxeuLxOxMOilYzVzEnWVFr79uLTj6vTPAPO2y2tbS
\$2a\$05\$MB.3MBazLhW0MRKzMBSzKOsHZE77Ou9mGW88yOG2Q5TxF5yvLe48C
\$2a\$05\$LBGyLBeuKxavLh.3MRWyLOhOiQVGYYyUBYE.rOUiRomCQFGIZfutEK
\$2a\$05\$LxiwMR.wKhGvKBiyKxeuM.rHYpUlrlOLksCsvo//8YT5J5z28rzBW
[...]

```
hashcat -m 3200 -a 0 hash.txt hint.txt
```

-> Time.Estimated : **2 hours**

```
hashcat -m 3200 -a 9 hash.txt hint.txt
```

-> Time.Estimated : **2 seconds**

Hashcat

<https://hashcat.net/forum/thread-9534.html>

Attack Mode (-a 9)

9 | Association : Semblable au mode single de John avec l'utilisation du username comme indice.

```
hashcat -m 0 -a 9 hash.txt hint.txt
```

hash.txt

```
6384e2b2184bcbf58eccf10ca7a6563c  
527bd5b5d689e2c32ae974c6229ff785  
b3725122c9d3bfef5664619e08e31877  
9d5e3ecdeb4cdb7acfd63075ae046672
```

hint.txt:

```
alice  
john  
marie  
kevin
```

-> Va effectuer des opérations de cassage par association entre les hash de chaque ligne avec les hints correspondants

Hashcat

<https://hashcat.net/forum/thread-9534.html>

Attack Mode (-a 9)

9 | Association : Semblable au mode single de John avec l'utilisation du username comme indice.

```
hashcat -m 0 -a 9 hash.txt hint.txt
```

hash.txt

```
6384e2b2184bcbf58eccf10ca7a6563c
527bd5b5d689e2c32ae974c6229ff785
b3725122c9d3bfef5664619e08e31877
9d5e3ecdeb4cdb7acfd63075ae046672
```

hint.txt:

```
alice
john
marie
kevin
```

-> Va effectuer des opérations de cassage par association entre les hash de chaque ligne avec les hints correspondants

⚠ J'ai testé et ça a pas marché, à voir si c'est pas encore trop récent comme mode

Hashcat

Options utiles

- `--username` : indique si le nom d'utilisateur est précisé en premier champ dans le fichier de hash (hashcat ne le détecte pas automatiquement)
- `--increment`, `--increment-min`, `--increment-max` : bruteforce incrémentiel avec des limites min et max
- `--outfile-autohex-disable` : empêche hashcat d'afficher le mot de passe au format \$HEX[09120BA0A9]
- `--encoding-to=` : permet de réencoder chaque candidat dans le format de charset que l'on souhaite (ex: `--encoding-to=utf-16le`)

```
hashcat -m 0 -a 0 hash.txt wordlist.txt --encoding-to=utf16be
```

- **--hex-charset** : indique que le masque fourni sera en hexadécimal (utile notamment lorsqu'on tente de casser des hash qui sont dans un encoding UTF-16LE par exemple)

[illegible]

Note: "utf16be" correspond à "UTF16 BigEndian", ce qui signifie que le caractère 'a' ne sera pas codé en hexa '0x61' mais '0x0061'. On peut donc construire un masque en hexa en rajoutant '00' devant chaque caractère.

Hashcat

Affiche les mots de passe cassés

```
hashcat -m 0 [-a 0 hash.txt wordlist.txt ou toute autre options] --show
```

Peu importe les options de cassages précisées

Comparaison

John ou Hashcat ?

4

John & Hashcat

Concept

⚠ Les formats de hash peuvent ou non être supportés par l'un ou l'autre outil selon des regex qui peuvent leur être propres

JOHN	HASHCAT
CPU	GPU
Plus simple d'utilisation (reconnaissance automatique de formats de hash)	Un peu plus complexe (fournir soi même le format du hash)
Rules++	Mask++
Rules fournies via le fichier de configuration john.conf	Rules fournies via l'option -r avec un fichier dédié .rule
<code>john hash.txt --show [--potfile=potfile]</code>	<code>hashcat -m [id] [options] hash.txt --show</code>
Détecte tout seul le username	On doit lui préciser si le username est présnet
<code>--pot=[potfile]</code>	<code>--potfile-path=[potfile]</code>
<code>\$HOME_JOHN/john.pot</code>	<code>\$HOME_HASHCAT/hashcat.potfile</code>

John & Hashcat

Potfile

- C'est le fichier qu'utilise l'outil pour stocker les hash cassés et le mot de passe correspondant
- 💡 Utile en mission de faire un potfile local dédié pour que chacun accède à ce qui a été cassé
- 💡 Pour vos vidéos d'exploitation, utile de faire un nouveau potfile avec une wordlist pour montrer une attaque de cassage de mots de passe
- C'est le potfile que l'on récupère lorsqu'on exporte les résultats d'un job sur la crackstation
- ⚠ Le nom d'utilisateur n'est pas dans le potfile (ce n'est pas nécessaire)
- Format d'un potfile: HASH:PASSWORD

JOHN	HASHCAT
--pot=[potfile]	--potfile-path=[porfile]
\$HOME_JOHN/john.pot	\$HOME_HASHCAT/hashcat.potfile

John & Hashcat

Quelques règles connues

Le fichier **w** contient "xmco".

- John

- Jumbo

```
john -w=w -ru=jumbo --stdout > w.jumbo
```

- Leet (external)

```
john -w=w -ex=leet --stdout > w.leet
```

- All

```
john -w=w -ru=all --stdout > w.all
```

john.conf

```
# External hybrid 'leet code  
[List.External:Leet]  
/*  
* 1337 language in this script:  
* a -> a4@  
* b -> b8  
* e -> e3  
* g -> g9  
* i -> i1!  
* l -> l1  
* o -> o0  
* s -> s$5  
* t -> t7  
*/
```

John & Hashcat

<https://github.com/hashcat/hashcat/tree/master/rules>

Quelques règles connues

- Hashcat

- OneRuleToRuleThemAll :

```
hashcat -a 0 w -r OneRuleToRuleThemAll.rule --stdout > w.1Rule2RuleThemAll
```

⚠ Peut générer beaucoup de duplicates

- Best66

```
hashcat -a 0 w -r best66.rule --stdout > w.best66
```

- Leetspeak

```
hashcat -a 0 w -r leetspeak.rule --stdout > w.leetspeak.rule
```

Wordlists

Wordlists les plus connues

- rockyou.txt : issu d'un leak de 2009 provenant de la société RockYou qui développait des widgets pour Facebook, MySpace, etc. 32 millions de mots de passe en clair ont été exposés à cause d'une injection SQL
- Seclists/* : énorme collection de listes contenant des mots de passe mais pas que : fuzzing web, noms d'utilisateurs, payloads, etc.)
- Wordlist d'openwall : Openwall c'est ceux qui maintiennent JohnTheRipper

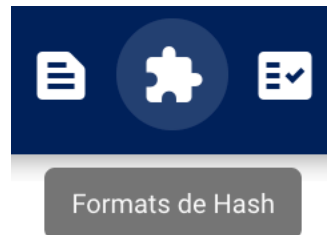
Liens

- <https://github.com/danielmiessler/SecLists/tree/master/Passwords> (contient rockyou.txt)
- <https://download.openwall.net/pub/wordlists/>

Un mot sur la crackstation

<https://crackstation.xmco.fr/>

- **Format de hash définis selon des regex**
 - **JWT** : format pris en charge uniquement côté Hashcat (donc pas besoin de créer des jobs John sinon ils ne se lanceront pas)
 - **Oracle11** : format pris en charge pour les deux outils avec la même regex



Edition du format de hash

^[A-Za-z0-9-_=]+\.[A-Za-z0-9-_=]+\.?[A-Za-z0-9-_.+/=]*\$

Regex

Nom	John	Hashcat	
JWT (JSON Web Tok	John	16500	

+

OK

Edition du format de hash

^(S:)?[a-fA-F0-9]{40}(:)?[a-fA-F0-9]{20}\$

Regex

Nom	John	Hashcat	
Oracle 11g/12c	oracle11	112	

+

OK

CTFD

-  Ne mettez pas vos hash sur la crackstation
-  Demandez pas à une IA le but c'est d'apprendre :)
-  Demandez-moi si vous êtes bloqués

CTFD

-  Ne mettez pas vos hash sur la crackstation
-  Évitez de demander à une IA
-  Demandez-moi si vous êtes bloqués

La Plateforme CTFD est exposée sur pentest-echo sur le port 65300

Sur votre MAC, exécutez :

```
ssh -L 65300:127.0.0.1:65300 pentest-echo.xmco.tech -p 42007
```

Puis allez sur <http://localhost:65300/>